



Classificação de Imagens de Plantas Doentes utilizando Visão Computacional

Henrique Mendonça Castelar Campos



Introdução

Um dos desafios encontrados na agricultura moderna consiste na identificação, detecção e isolamento de culturas de plantas doentes, visto que a doença pode se espalhar, resultando na contaminação de toda a produção agrícola

Este trabalho tem como objetivo o desenvolvimento de um modelo de rede neural convolucional capaz de utilizar visão computacional para ajudar a identificar plantas doentes.



Trabalhos Relacionados

- Bahaa S. Hamed, Mahmoud M. Hussein, Afaf M. Mousa em [1] propuseram o desenvolvimento de uma rede neural convolucional EfficientNetV2S para a classificação de doenças em plantas.
- Em [2], foi proposto o desenvolvimento de uma plataforma chamada “FarmEasy” na qual possui capacidade de detectar e classificar doenças de plantas.



Metodologia

Dataset:

- Plant Village Dataset [3]
- 38 classes de doenças de plantas, incluindo classes de plantas saudáveis e doentes



Metodologia

Pré-Processamento

- Criação de uma arquivo no formato CSV com os seguintes atributos:
 - Nomes dos arquivos de imagem
 - Versão do arquivo (colorido, escalas de cinza e segmentado)
 - Tipo de doença
- Separação do conjunto de treino e teste
 - 20% para o conjunto de teste
 - Estado randômico: 1024



Metodologia

Modelos desenvolvidos

- ResNet50
- EfficientNetV2S
- GoogLeNet



Implementação

ResNet50

```
resnet50.py

import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

...

# Construindo modelo ResNet50
base_model = tf.keras.applications.ResNet50(
    weights = 'imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

# Congelando as camadas base para nao serem atualizadas no primeiro momento
base_model.trainable = False

# Construindo modelo final
inputs = tf.keras.Input(shape=(224, 224, 3))

x = base_model(inputs, training=False)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.2)(x)

# Camada de saida com neurônios igual ao número de classes
outputs = tf.keras.layers.Dense(num_classes, activation='softmax')(x)

# Construindo modelo
model = tf.keras.Model(inputs, outputs)

...
```



Implementação

EfficientNetV2S

```
efficientnetv2s.py

import tensorflow as tf
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.applications import EfficientNetV2S
from tensorflow.keras import models, layers

...

# Construindo modelo EfficientNetV2S
base_model = EfficientNetV2S(
    include_top=False,
    weights='imagenet',
    input_shape=(224, 224, 3)
)

# Congelando as camadas base para nao serem atualizadas no primeiro momento
base_model.trainable = False

# Construindo o modelo sequencial adicionando a base e as novas camadas de classificacao
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.2),
    layers.Dense(num_classes, activation='softmax')
])

...
```


Implementação

GoogLeNet

```
googlenet.py

import tensorflow as tf
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.layers import (Conv2D, MaxPooling2D, AveragePooling2D, Dropout, Dense, Flatten, Input, concatenate)
from tensorflow.keras.models import Model

...

# Construindo modelo GoogLeNet
def inception_module(x, filters_1x1, filters_3x3_reduce, filters_3x3, filters_5x5_reduce, filters_5x5, filters_pool_proj,
name=None):
    # Ramo 1: Convolucao 1x1
    path1 = Conv2D(filters_1x1, (1, 1), padding='same', activation='relu', name=f'{name}_1x1')(x)

    # Ramo 2: Convolucao 1x1 → Convolucao 3x3
    path2 = Conv2D(filters_3x3_reduce, (1, 1), padding='same', activation='relu', name=f'{name}_3x3_reduce')(x)
    path2 = Conv2D(filters_3x3, (3, 3), padding='same', activation='relu', name=f'{name}_3x3')(path2)

    # Ramo 3: Convolucao 1x1 → Convolucao 5x5
    path3 = Conv2D(filters_5x5_reduce, (1, 1), padding='same', activation='relu', name=f'{name}_5x5_reduce')(x)
    path3 = Conv2D(filters_5x5, (5, 5), padding='same', activation='relu', name=f'{name}_5x5')(path3)

    # Ramo 4: MaxPool 3x3 → Convolucao 1x1
    path4 = MaxPooling2D((3, 3), strides=(1, 1), padding='same', name=f'{name}_pool')(x)
    path4 = Conv2D(filters_pool_proj, (1, 1), padding='same', activation='relu', name=f'{name}_pool_proj')(path4)

    # Concatenacao dos ramos na dimensao dos canais
    return concatenate([path1, path2, path3, path4], axis=-1, name=name)

def auxiliary_classifier(x, name, num_classes):
    # Reducao espacial
    x = AveragePooling2D((5, 5), strides=(3, 3), name=f'{name}_avgpool')(x)
    x = Conv2D(128, (1, 1), padding='same', activation='relu', name=f'{name}_conv')(x)
    x = Flatten(name=f'{name}_flatten')(x)
    x = Dense(1024, activation='relu', name=f'{name}_fc')(x)
    x = Dropout(0.7, name=f'{name}_dropout')(x)
    x = Dense(num_classes, activation='softmax', name=f'{name}_output')(x)

    return x
```

Implementação

GoogLeNet

```
def build_googlenet(input_shape, num_classes):
    input_layer = Input(shape=input_shape)

    # Estagio 1
    x = Conv2D(64, (7, 7), strides=(2, 2), padding='same', activation='relu', name='conv1_7x7')(input_layer)
    x = MaxPooling2D((3, 3), strides=(2, 2), padding='same', name='maxpool1_3x3')(x)

    # Estagio 2
    x = Conv2D(64, (1, 1), padding='same', activation='relu', name='conv2_1x1')(x)
    x = Conv2D(192, (3, 3), padding='same', activation='relu', name='conv2_3x3')(x)
    x = MaxPooling2D((3, 3), strides=(2, 2), padding='same', name='maxpool2_3x3')(x)

    # Estagio 3 (Inception 3a e 3b)
    x = inception_module(x, 64, 96, 128, 16, 32, 32, name='inception_3a')
    x = inception_module(x, 128, 128, 192, 32, 96, 64, name='inception_3b')
    x = MaxPooling2D((3, 3), strides=(2, 2), padding='same', name='maxpool3_3x3')(x)

    # Estagio 4 (Inception 4a a 4e) + Classificador Auxiliar 1
    x = inception_module(x, 192, 96, 208, 16, 48, 64, name='inception_4a')
    aux1 = auxiliary_classifier(x, name='aux1', num_classes=num_classes)

    x = inception_module(x, 160, 112, 224, 24, 64, 64, name='inception_4b')
    x = inception_module(x, 128, 128, 256, 24, 64, 64, name='inception_4c')
    x = inception_module(x, 112, 144, 288, 32, 64, 64, name='inception_4d')
    aux2 = auxiliary_classifier(x, name='aux2', num_classes=num_classes)

    x = inception_module(x, 256, 160, 320, 32, 128, 128, name='inception_4e')
    x = MaxPooling2D((3, 3), strides=(2, 2), padding='same', name='maxpool4_3x3')(x)

    # Estagio 5 (Inception 5a e 5b)
    x = inception_module(x, 256, 160, 320, 32, 128, 128, name='inception_5a')
    x = inception_module(x, 384, 192, 384, 48, 128, 128, name='inception_5b')

    # Classificador principal
    x = AveragePooling2D((7, 7), strides=(1, 1), name='global_avg_pool')(x)
    x = Flatten(name='final_flatten')(x)
    main_output = Dense(num_classes, activation='softmax', name='main_output')(x)

    # O modelo retorna 3 saidas (1 principal e 2 auxiliares)
    model = Model(inputs=input_layer, outputs=[main_output, aux1, aux2], name='GoogLeNet')
    return model

# Construindo modelo
model = build_googlenet(input_shape=(224, 224, 3), num_classes=num_classes)

...
```

ResNet50

Matriz de Confusão - ResNet (PlantVillage)

[illegible]

Classe Prevista (Predição)



Resultados

ResNet50

Média da Taxa de Verdadeiro Positivo (TVP): 97,82977590360394%

Média da Taxa de Verdadeiro Negativo (TVN): 99,9515011065704%

Média da Taxa de Falso Positivo (TFP): 00,04849889342958133%

Média da Taxa de Falso Negativo (TFN): 02,170224096396056%

Acurácia: 98,21691627792781%

Perda: 5,45535683631897%

Matriz de Confusão - GoogleNet (PlantVillage)

Classe Real (Verdadeiro)

Classe Prevista (Predição)



Resultados

GoogLeNet

Média da Taxa de Verdadeiro Positivo (TVP): 96,82533375121448%

Média da Taxa de Verdadeiro Negativo (TVN): 99,93167297621002%

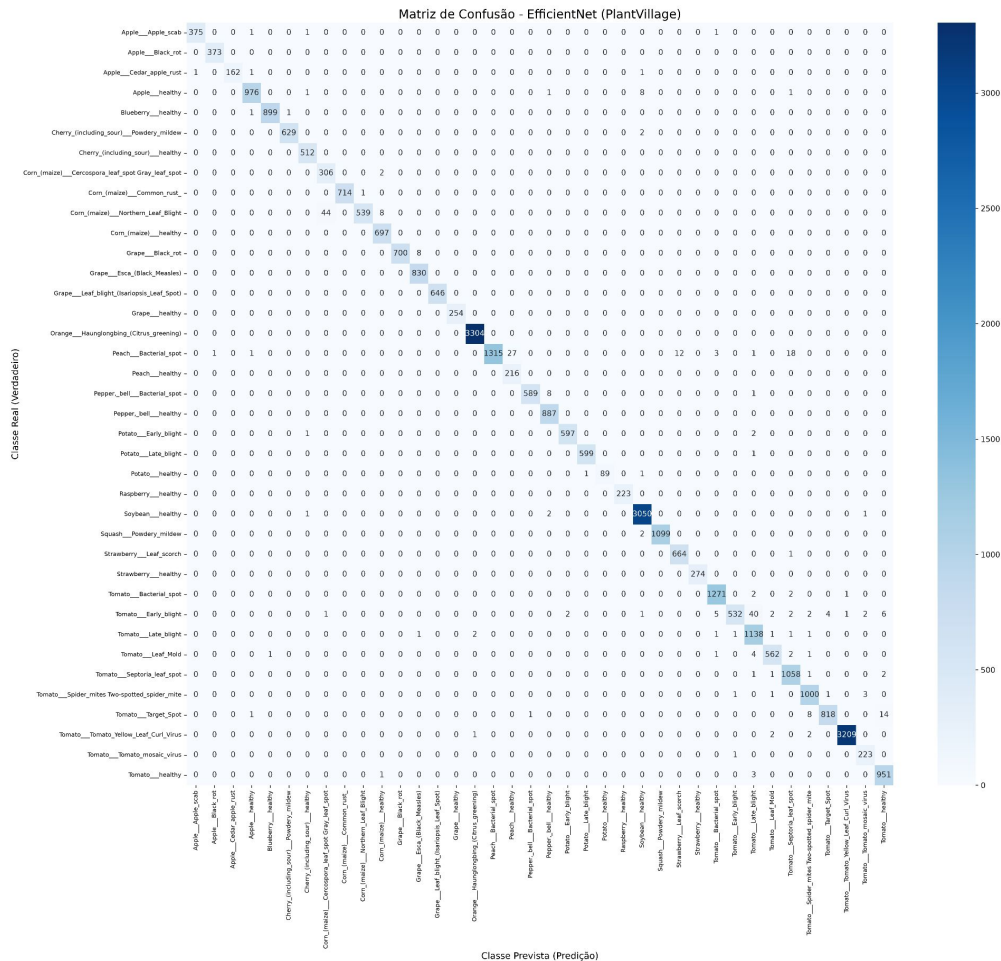
Média da Taxa de Falso Positivo (TFP): 00,06832702378997811%

Média da Taxa de Falso Negativo (TFN): 03,174666248785538%

Acurácia: 97,80752062797546%

Perda: 07,479541748762131%

EfficientNetV2S





Resultados

EfficientNetV2S

Média da Taxa de Verdadeiro Positivo (TVP): 98,85987305127136%

Média da Taxa de Verdadeiro Negativo (TVN): 99,97472136956506%

Média da Taxa de Falso Positivo (TFP): 00,02527863043493335%

Média da Taxa de Falso Negativo (TFN): 01,1401269487286432%

Acurácia: 99,06702676160078%

Perda: 02,663467451930046%



Conclusão

O modelo **EfficientNetV2S** foi o que obteve a melhor acurácia quando comparado ao **GoogLeNet** e ao **ResNet50** para o Dataset PlantVillage.



Referências

[1] HAMED, Bahaa S.; HUSSEIN, Mahmoud M.; MOUSA, Afaf M. Plant disease detection using deep learning. International Journal of Intelligent Systems and Applications, v. 15, n. 6, p. 38-50, 2023.

[2] P. Pandey, K. Patyane, M. Padekar, R. Mohite, P. Mane and A. Avhad, "Plant Disease Detection Using Deep Learning Model - Application FarmEasy," 2023 International Conference on Advanced Computing Technologies and Applications (ICACTA), Mumbai, India, 2023, pp. 1-6, doi: 10.1109/ICACTA58201.2023.10393095. keywords: {Deep learning;Training;Plant diseases;Computer vision;Machine learning algorithms;Automation;Manuals;Machine learning;Plant disease;Image analysis;Convolutional neural networks;Disease detection;Precision agriculture;Feature extraction},

[3] PLANTVILLAGE Dataset. Kaggle, 2019. Disponível em: <https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset>. Acesso em: 18 jun. 2026.